EE1007
Introduction to Computer Science Laboratory

Lab 7
Functions

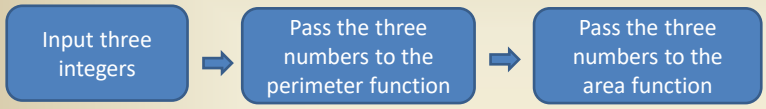
Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw

Lab 7.1
P113025 Triangle (30%)

- Test target: functions
- You need to write two functions to calculate the perimeter and area of a triangle. One function should calculate the perimeter, and the other should calculate the area.
- You needs to design fool-proof to avoid program errors. If three numbers cannot form a triangle, print ”error”.

Lab 7.1
P113025 Triangle (30%)

- Flow chart:



void perimeter (sides){
    adder;
}

void area (sides){
    Heron's formula
}

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

```
void trianglePerimeter(int x, int y, int z) {
    cout << "perimeter: " << x + y + z << endl;
}
```

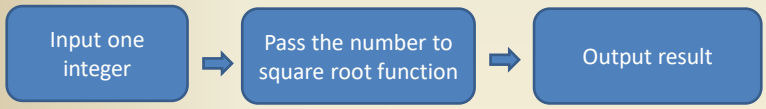
```
void triangleArea(int x, int y, int z) {
    cout << "area: " << sqrt((x + y + z) / 2 * (x - y + z) / 2 * (x + y - z) / 2 * (y - x + z) / 2)
}
```


Lab 7.2
P113026 Oil Lamp (30%)

- Test target: functions
- You need to write one function to calculate the result. Otherwise, you will get 0 point.
- The program don't need to design fool-proof to avoid program errors.

Lab 7.2
P113026 Oil Lamp (30%)

◆ Flow chart:



Lab 7.2
P113026 Oil Lamp (30%)

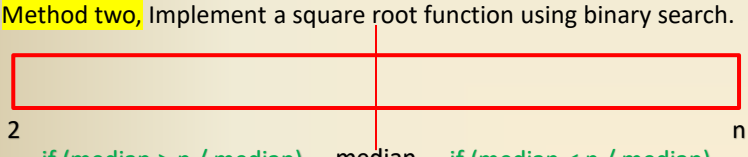
◆ How to know whether the final light is on or off?

If the number of factors is odd, the light will be on.

➢ Whether the number is a perfect square.

Method one, Use sqrt and convert it to long long int using static_cast.

Method two, Implement a square root function using binary search.




```
if (median > n / median)
    median--;
if (median < n / median)
    median++;
```

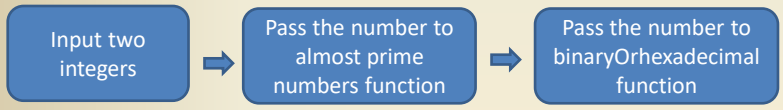
```
if (median == n / median && n % median == 0)
    return true;
```

Lab 7.3
P113027 Almost Prime Numbers (30%)

- Test target: functions
- You need to write at least one function to calculate the result.
- No array, vector, and string is allowed.
- The program don't need to design fool-proof to avoid program errors.

Lab 7.3
P113027 Almost Prime Numbers (30%)

- Flow chart:



Lab 7.3
P113027 Almost Prime Numbers (30%)

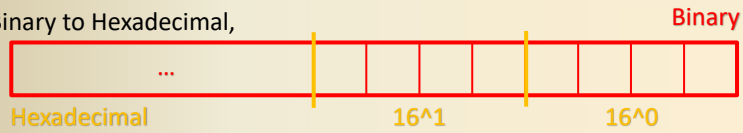
- How many almost prime numbers are there in the range?
  Check for factors from 2 to sqrt(n).
- Convert decimal to binary or hexadecimal.

  Binary,

  Divide by 2: Divide the decimal number by 2 and note the quotient and the remainder.

  Repeat: Continue dividing the quotient by 2 until the quotient is 0, and record the remainder at each step.

Binary to Hexadecimal,



Lab 7.3
P113027 Almost Prime Numbers (30%)

```
//16進位
if (origin >= 16) {
    cout << " hexadecimal: ";

    while (counter > 0) {
        int decimal = 0, base = 1;
        long long int temp;

        if (counter % 4 != 0) {
            temp = binary / (int)pow(10, counter / 4 * 4);
            binary %= (int)pow(10, counter / 4 * 4);
            counter -= (counter % 4);
        } else {
            temp = binary / (int)pow(10, counter - 4);
            binary %= (int)pow(10, counter - 4);
            counter -= 4;
        }

        while (temp > 0) {
            int last_digit = temp % 10;
            temp /= 10;
            decimal += last_digit * base;
            base *= 2;
        }

        if (decimal == 10)
            cout << "A";
        else if (decimal == 11)
            cout << "B";
        else if (decimal == 12)
            cout << "C";
        else if (decimal == 13)
            cout << "D";
        else if (decimal == 14)
            cout << "E";
        else if (decimal == 15)
            cout << "F";
        else
            cout << decimal;
    }

    cout << endl;
}
```

Take a 4-digit number sequentially.

binary -> decimal

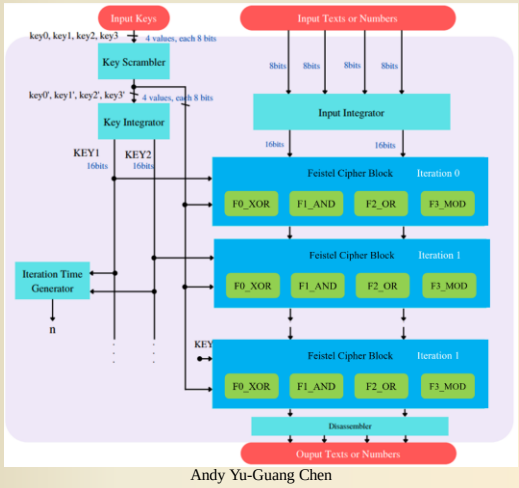
output

Lab 7.4
P113028 Feistel Cipher (20%)

- Test target: functions
- You need to write at least one function to calculate the result.
- No array, vector, and string is allowed.
- The program don't need to design fool-proof to avoid program errors.

Lab 7.4
P113028 Feistel Cipher (20%)

- Flow chart:

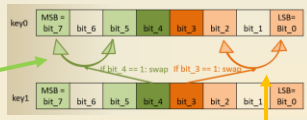


Lab 7.4
P113028 Feistel Cipher (20%)

- Step 1, 2, 3

  Input data.

- Step 4
  - Convert decimal to binary. Refer to lab 7.3.
  - Write a function to perform a swap.




```
int SWAP(int x, int y) {
    if ((x % 100000 - x % 10000) / 10000) {
        int temp = y;
        y = y + (y % 1000000 - y % 100000) * 100 - (y % 1000000 - y % 100000);
        y = y + (temp % 100000000 - temp % 10000000) / 100 - (temp % 100000000 - temp % 10000000);
    }

    if (((x % 10000 - x % 1000) / 1000)) {
        int temp = y;
        y = y + (y % 10) * 100 - (y % 10);
        y = y + (temp % 1000 - temp % 100) / 100 - (temp % 1000 - temp % 100);
    }

    return y;
}
```


Lab 7.4
P113028 Feistel Cipher (20%)

- Step 5

  Concatenate key0' with key1' and key2' with key3' using addition and multiplication.

- Step 6, 7
  - Use a number to record the current iteration count.
  - Determine the KEY, key', and function to be used for this iteration.

    XOR: ^ AND: & OR: | MOD: You need to write it yourself.

```
int SWITCH(int x, int R0, int KEY, int L0) {
    if (x == 0)
        R0 = R0 ^ KEY ^ L0;
    else if (x == 2)
        R0 = (R0 | KEY)^L0;
    else if (x == 1)
        R0 = (R0 & KEY)^L0;
    else {
        long long p = to_binary(R0), q = to_binary(KEY);
        int n = ((to_decimal(p % 100000000)) % (to_decimal(q % 100000000)));
        int m = ((to_decimal(p / 100000000)) % (to_decimal(q / 100000000)));
        R0 = L0 ^ to_decimal(to_binary(m) * 100000000 + to_binary(n));
    }

    return R0;
}
```

Evaluation

- Lab7.1(30%)
  - 5 testcases, 6% each
- Lab7.2(30%)
  - 5 testcases, 6% each
- Lab7.3(30%)
  - 5 testcases, 6% each
- Lab7.4(20%)
  - 5 testcases, 4% each





Andy, Yu-Guang Chen

Assistant Professor, Department of EE, NCU

Email: andyygchen@ee.ncu.edu.tw